13.3 Hyperparameters

What you tune, and how you choose it.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

Figures and notes follow Géron, *Hands-On Machine Learning*; Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*; and Theodoridis.

1. Number of hidden layers

Many problems work with **one or two** hidden layers. MNIST: a few hundred neurons in one hidden layer can pass 97%; two hidden layers with the same total neuron count can pass 98%, in about the same training time.

For harder tasks, add layers until the training set starts to **overfit**. Large image classification and speech often want dozens of layers (sometimes hundreds), not fully connected, and a lot of data.

You rarely train those from scratch. **Reuse** part of a pretrained net for a similar task: faster, less data.

---

2. Neurons per hidden layer

Input and output sizes are fixed by the task. MNIST: **784** inputs, **10** outputs.

Hidden widths used to be a **pyramid** (fewer neurons as you go up: many low-level features collapse into fewer high-level ones). That fashion has faded.

As with depth, grow width until you overfit. You usually get more from **another layer** than from more neurons in the layers you already have. There is still no closed-form "right" width.

A simpler move: pick **more** layers and neurons than you need, then **early stopping** (and dropout, and other regularizers) to shrink the effective model. **Stretch pants**: do not hunt for a perfect size; start large and let the regularizer take them in.

---

3. Learning rate, batch size, and the rest

Learning rate is the hyperparameter that most often makes or breaks training. A useful target is about **half** the largest rate that still diverges. Hunt: start too large, divide by 3 until divergence stops.

Use a better optimizer than plain mini-batch GD, and tune *its* hyperparameters too.

Batch size changes both quality and wall time. Typical good sizes are **under 32**. Larger batches give a cleaner gradient, but the landscape is messy enough that this often does not help. Sizes **above 10** use hardware (matrix multiplies) better. **Batch normalization** wants batches not smaller than about 20.

Activation: ReLU is a sound default for every hidden layer. The output activation depends on the task.

Number of iterations: usually do not tune it. Use **early stopping**.

---

4. Searching the combination

Try combinations; keep what wins on **validation** (or K-fold).

Randomized search is enough for many small problems. When training is slow, you only see a tiny slice of the space. Help it by hand:

1. A quick random search on **wide** ranges.
2. Another search on **narrower** ranges around the winners.
3. Repeat.

---

5. Libraries

Tool	Role
Hyperopt	Search over mixed spaces (rates, depths, ...)
Hyperas, **kopt**, **Talos**	Keras models (the first two sit on Hyperopt)
Scikit-Optimize (skopt)	General; `BayesSearchCV` is a Bayesian cousin of `GridSearchCV`
Spearmint	Bayesian optimization
Sklearn-Deap	Evolutionary search, GridSearchCV-like interface

---

Practice

1. What is the “stretch pants” approach to depth and width?
2. You can afford a little more capacity. According to this lecture, do you add a hidden layer or add neurons to the layers you already have?
3. Describe the divide-by-3 hunt for a learning rate.